Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение высшего образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Отчет

по лабораторной работе №3 «Строки и ввод-вывод»

по дисциплине «Программирование»

Вариант 1-2-2-4

Факультет: Безопасность информационных технологий

Группа: N3149

Выполнила: Беленькая Полина Александровна

Подпись:

Преподаватель: Гирик Алексей Валерьевич

Санкт-Петербург 2024

Задание

Разработать на языке C для ОС Linux программу, которая читает данные из стандартного потока ввода или файла, осуществляет преобразование в соответствии с вариантом лабораторной работы и выводит результат в стандартный поток вывода или файл. Данные для поиска: IPv4-адреса. Опции, с помощью которых задается ограничение преобразования: -f=M, -t=N. Разметка Markdown, используемая для выделения найденных фрагментов текста: жирное начертание. Цвет для выделения найденных фрагментов текста при выводе с опцией -c: синий.

Makefile

```
  GNU nano 7.2                                          Makefile
.PHONY: all clean

APP=prg3pabN3149
CFLAGS=-Wall -Wextra -Werror -g

all: $(APP)

$(APP): $(APP).c
        gcc -o $(APP) $(CFLAGS) $(APP).c

clean:
        rm $(APP)
```

Примеры работы программ на различных исходных данных:

```
polina@DESKTOP-QBMU2N6:~$ ./prg3pabN3149 -x in.txt out.tx
./prg3pabN3149: invalid option -- 'x'
```

```
polina@DESKTOP-QBMU2N6:~$ ./prg3pabN3149
Нажмите Ctrl + D для завершения.
IPv4 адрес: 255.188.12.0
привет
неверный адрес:  255.188.12.0.2 и 350.22.2.1
верные адреса: 255.255.255.0 и 240.39.0.0

IPv4 адрес: **255.188.12.0**
привет
неверный адрес:  255.188.12.0.2 и 350.22.2.1
верные адреса: **255.255.255.0** и **240.39.0.0**
polina@DESKTOP-QBMU2N6:~$
```

```
polina@DESKTOP-QBMU2N6:~$ ./prg3pabN3149 -f=2 -t=3
Нажмите Ctrl + D для завершения.
IPv4 адрес: 255.188.12.0
привет
неверный адрес:  255.188.12.0.2 и 350.22.2.1
верные адреса: 255.255.255.0 и 240.39.0.0 и 254.34.3.1

IPv4 адрес: 255.188.12.0
привет
неверный адрес:  255.188.12.0.2 и 350.22.2.1
верные адреса: **255.255.255.0** и **240.39.0.0** и 254.34.3.1
```

```
polina@DESKTOP-QBMU2N6:~$ ./prg3pabN3149 -v
Беленькая Полина Александровна, гр. N3149.
Вариант 1-2-2-4.
```

```
polina@DESKTOP-QBMU2N6:~$ ./prg3pabN3149 -f=2 -t=3 input.txt
Данные из файла:
 192.168.0.1
10.0.0.256
172.16.254.1
255.255.255.255
192.168.1.300
0.0.0.0
::1
2001:0db8:85a3:0000:0000:8a2e:0370:7334


192.168.0.1
10.0.0.256
**172.16.254.1**
**255.255.255.255**
192.168.1.300
0.0.0.0
::1
2001:0db8:85a3:0000:0000:8a2e:0370:7334
```

```
polina@DESKTOP-QBMU2N6:~$ ./prg3pabN3149 -c -f=2 input.txt
Данные из файла:
 192.168.0.1
10.0.0.256
172.16.254.1
255.255.255.255
192.168.1.300
0.0.0.0
::1
2001:0db8:85a3:0000:0000:8a2e:0370:7334


192.168.0.1
10.0.0.256
172.16.254.1
255.255.255.255
192.168.1.300
0.0.0.0
::1
2001:0db8:85a3:0000:0000:8a2e:0370:7334
```

```
polina@DESKTOP-QBMU2N6:~$ ./prg3pabN3149 input.txt output.txt
Данные из файла:
 192.168.0.1
10.0.0.256
172.16.254.1
255.255.255.255
192.168.1.300
0.0.0.0
::1
2001:0db8:85a3:0000:0000:8a2e:0370:7334
Преображённые данные сохранены в файл успешно.
```

Исходный текст программы

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <locale.h>
#include <ctype.h>
#include <unistd.h>

#define BLUE "\x1b[34m"
#define COLOR_RES "\x1b[0m"

char *inp;

void normalize_line_endings(char *str) {
    char *src = str;  // Указатель для чтения
    char *dst = str;  // Указатель для записи

    while (*src) {
```

```
        if (*src == '\r' && *(src + 1) == '\n') {
            *dst = '\n';  // Заменить \r\n на \n
            src += 2;     // Пропустить \r\n
        } else {
            *dst = *src;  // Копировать обычный символ
            src++;
        }
        dst++;
    }
    *dst = '\0';  // Завершить строку нулевым символом
}


int is_valid_ipv4(const char *start, const char *end);

char *mark_ipv4(const char *inp, int is_c, int f, int t, int is_n) {
    size_t len = strlen(inp);
            (void)is_n;  // Пометка для подавления предупреждения
    const char *start = inp;  // Начало текущего блока текста
    const char *cur = inp;    // Текущий символ
    char *res = calloc(len * 2, sizeof(char)); // Результирующая строка
    if (!res) {
        perror("Не удалось выделить память.");
        exit(2);
    }

    int valid_count = 0; // Счётчик найденных объектов, соответствующих критериям

    while (*cur) {
        if (isdigit(*cur)) {
            const char *ipv4_start = cur;
            while (isdigit(*cur) || *cur == '.') cur++;
            const char *ipv4_end = cur - 1;

            if (is_valid_ipv4(ipv4_start, ipv4_end)) {
                valid_count++;

                // Проверяем, входит ли текущий объект в диапазон
                if (valid_count >= f && (t == -1 || valid_count <= t)) {
                    // Добавляем текст до IPv4-адреса
                    strncat(res, start, ipv4_start - start);

                    // Добавляем корректный IPv4-адрес с выделением
                    if (is_c && valid_count >= f) { // Выделяем цветом начиная с f-го адреса
                        strcat(res, BLUE);
                        strncat(res, ipv4_start, ipv4_end - ipv4_start + 1);
                        strcat(res, COLOR_RES);
                    } else {
                        strcat(res, "**");
                        strncat(res, ipv4_start, ipv4_end - ipv4_start + 1);
                        strcat(res, "**");
                    }

                    start = cur; // Обновляем начало следующего блока
                }
            }
        } else {
            cur++;
        }
    }

    // Добавляем оставшийся текст
    strcat(res, start);
    return res;
}
```

```
int is_valid_ipv4(const char *start, const char *end) {
    int octet = 0, octet_count = 0;
    const char *ptr = start;

    while (ptr <= end) {
        if (isdigit(*ptr)) {
            octet = octet * 10 + (*ptr - '0');
            if (octet > 255) {
                return 0; // Октет больше 255
            }
        } else if (*ptr == '.') {
            octet_count++;
            if (octet_count > 3) {
                return 0; // Слишком много точек
            }
            octet = 0; // Сброс значения для нового октета
        } else {
            return 0; // Недопустимый символ
        }
        ptr++;
    }

    // После последнего октета
    octet_count++; // Учитываем последний октет
    if (octet_count != 4) {
        return 0; // Должно быть ровно 4 октета
    }

    return 1; // Адрес валиден
}

// Изменения в обработке опций командной строки

int main(int argc, char *argv[]) {
    setlocale(LC_ALL, "ru_RU.UTF-8");
    char *DEBUG = getenv("LAB3DEBUG");
    if (DEBUG) {
        fprintf(stderr, "Включен вывод отладочных сообщений.\n");
    }

    char *opts = "vcnf:t:";
    int opt;
    int c_flag = 0, n_flag = 0, f_flag = 0, t_flag = 0;
    int t_val = -1;
    int f_val = -1;
    while ((opt = getopt(argc, argv, opts)) != -1) {
        switch (opt) {
            case 'v':
                fprintf(stdout, "Беленькая Полина Александровна, гр. N3149.\n");
                fprintf(stdout, "Вариант 1-2-2-4.\n");
                exit(0);
                break;
            case 'c':
                c_flag = 1;
                break;
            case 'n':
                n_flag = 1;
                break;
            case 'f':
                if (optarg != NULL && optarg[0] == '=') {
                    if (sscanf(optarg + 1, "%d", &f_val) != 1) {
                        fprintf(stderr, "Неверное значение для -t. Используйте числа.\n");
                        return EXIT_FAILURE;
                    }
                } else if (optarg != NULL) {
                    if (sscanf(optarg, "%d", &f_val) != 1) {
```

```
                fprintf(stderr, "Неверное значение для -f. Используйте числа.\n");
                return EXIT_FAILURE;
            }
        } else {
            fprintf(stderr, "Опция -f требует аргумент.\n");
            return EXIT_FAILURE;
        }
        f_flag = 1;
        break;

    case 't':
        if (optarg != NULL && optarg[0] == '=') {
            if (sscanf(optarg + 1, "%d", &t_val) != 1) {
                fprintf(stderr, "Неверное значение для -t. Используйте числа.\n");
                return EXIT_FAILURE;
            }
        } else if (optarg != NULL) {
            if (sscanf(optarg, "%d", &t_val) != 1) {
                fprintf(stderr, "Неверное значение для -t. Используйте числа.\n");
                return EXIT_FAILURE;
            }
        } else {
            fprintf(stderr, "Опция -t требует аргумент.\n");
            return EXIT_FAILURE;
        }
        t_flag = 1;
        break;

    default:
        break;
    }
}

int fin = -1;
int fout = -1;

if (f_val > t_val && t_flag == 1) {
    fprintf(stderr, "Ошибка: t должно быть больше чем f.\n");
    exit(4);
}
if (argc - optind > 2) {
    fprintf(stderr, "Слишком много аргументов.\n");
    exit(4);
} else if (argc - optind == 2) {
    fin = optind;
    fout = optind + 1;
} else if (argc - optind == 1) {
    fin = optind;
}

if (fin == -1) {
    size_t buf = 100;
    inp = calloc(buf, sizeof(char));
    int counter = 0;
    char ch;
    if (!inp) {
        perror("Проблемы с памятью: ");
        exit(2);
    }
    fprintf(stdout, "Нажмите Ctrl + D для завершения.\n");
    while ((ch = getchar()) != EOF) {
        if (ch == '\n') {
            inp[counter++] = ch;
            continue;
        }
        inp[counter++] = ch;
```

```
            if ((size_t)(counter + 10) > buf) {
                buf *= 2;
                char *new_inp = realloc(inp, buf * sizeof(*inp));
                if (!new_inp) {
                    free(inp);
                    perror("Проблемы с памятью: ");
                    exit(2);
                }
                inp = new_inp;
            }
        }
    } else {
        FILE *f = fopen(argv[fin], "r");
        if (!f) {
            perror("Ошибка чтения файла.");
            exit(3);
        }
        fseek(f, 0, SEEK_END);
        long fsize = ftell(f);
        rewind(f);
        if (fsize == -1) {
            perror("Ошибка при определении размера файла.");
            fclose(f);
            exit(3);
        }
        free(inp);
        inp = calloc(fsize + 1, sizeof(char));
        if (!inp) {
            perror("Проблемы с памятью: ");
            exit(2);
        }

        size_t rsize = fread(inp, sizeof(char), fsize, f);
        if ((size_t)rsize != (size_t)fsize) {
            perror("Ошибка при чтении файла.");
            fclose(f);
            exit(3);
        }
        inp[fsize] = '\0';
        normalize_line_endings(inp);

        fclose(f);
        fprintf(stdout, "Данные из файла:\n %s", inp);
    }

    if (!f_flag) f_val = 1;
    if (!t_flag) t_val = strlen(inp);
    char *res = mark_ipv4(inp, c_flag, f_val, t_val, n_flag);

    if (fout == -1) {
        fprintf(stdout, "\n\n%s\n", res);
    } else {
        FILE *out = fopen(argv[fout], "a");
        if (fputs(res, out) == EOF) {
            perror("Ошибка записи преображённых данных в файл.\n");
            fclose(out);
            exit(3);
        }
        fprintf(stdout, "Преображённые данные сохранены в файл успешно.\n");
        fclose(out);
    }

    return EXIT_SUCCESS;
}
```